

Revision — 1.4.2 Data Structures

OCR H446 — one-page revision summary

Must know

- **Static vs dynamic:** static = size fixed at declaration, contiguous block, fast access but may waste/run out of memory (e.g. **array**). Dynamic = grows/shrinks at run time, allocated from the **heap**, flexible but has pointer overhead (e.g. **linked list**).
- **Array:** fixed size, **same type**, index-accessed (0-based), $O(1)$ access. 2D `grid[row][col]`, 3D `cube[layer][row][col]`. **Record:** named fields of **different** types (`student.name`). **Tuple:** ordered, **immutable**. **List:** ordered, mutable, resizable.
- **Linked list:** **nodes** = data + pointer to next; head pointer, last node = null. Easy insert/delete by repointing; **no random access** — must traverse from the head.
- **Stack (LIFO):** push/pop/peek at the **top** via a stack pointer; overflow/underflow. Uses: call stack, undo, backtracking.
- **Queue (FIFO):** enqueue at the **rear**, dequeue at the **front**. **Circular:** `rear = (rear + 1) MOD size` reuses freed space. **Priority queue:** highest priority served first. Uses: print spooling, buffers, scheduling.
- **Graph:** vertices + edges; directed/undirected/weighted. **Adjacency matrix** = dense, $O(1)$ lookup; **adjacency list** = sparse, memory-efficient.
- **Tree:** connected, acyclic; root, parent/child, leaf. **Binary tree** ≤ 2 children per node.
- **BST:** binary tree where **left subtree** < **node** < **right subtree**; insert by going left if smaller / right if larger; $O(\log n)$ search when balanced. A BST is **not** just any binary tree — it needs the ordering property.
- **Traversals:** **pre-order** (root, L, R) — copy/prefix; **in-order** (L, root, R) — **BST gives ascending order**; **post-order** (L, R, root) — delete/postfix.
- **Hash table:** key $\rightarrow$ index via a **hash function** (e.g. `key MOD size`) for $\sim O(1)$ lookup. **Collision** = two keys map to the same index $\rightarrow$ resolve by linear probing or chaining. Higher load factor = more collisions = slower.

Key terms

Term	Definition
Array	Ordered, fixed-size, same-type collection accessed by index
Record	Collection of named fields that may hold different data types
Tuple	Ordered, immutable sequence
Linked list	Dynamic list of nodes, each = data + pointer to the next
Stack	LIFO structure; push/pop/peek at the top
Queue	FIFO structure; enqueue at rear, dequeue at front
BST	Binary tree where left subtree < node < right subtree
Hash table	Maps keys to indices via a hash function; collision = 2 keys, same index

Worked example / how to

Trace a stack (top listed first), starting empty: `push(5) → [5]` · `push(9) → [9, 5]` · `push(2) → [2, 9, 5]` `pop()` → returns **2** → `[9, 5]` · `push(7) → [7, 9, 5]` · `pop()` → returns **7** → `[9, 5]` **Popped in order: 2, then 7. Final stack (top → bottom): 9, 5.** (LIFO — the most recently pushed value is always popped first.)

Exam tips

- Name **LIFO vs FIFO** ends correctly: stack push/pop at the **top**; queue enqueue **rear**, dequeue **front**.
- A **BST** needs the left < node < right ordering — don't confuse it with any binary tree. In-order traversal gives ascending order.
- For static vs dynamic, quote both **size** (fixed vs run-time) and **memory** (contiguous vs heap) differences.
- Match structure to use case: Back button = **stack** (most recent first); buffer/spooling = **queue** (arrival order).